MERN Expense Tracker

Full-Stack Web Application

Built with MongoDB · Express.js · React.js · Node.js

This document is the project report for the MERN Expense Tracker, a full-stack web application that allows users to record, view, edit, delete, filter, and analyze their daily income and expense transactions. The report covers all features, the technology stack, folder structure, API endpoints, setup instructions, and usage examples, drawn directly from the project README.

Features

The application provides a complete set of features for managing personal finances through a clean, responsive dashboard. The following table summarizes each feature and its purpose:

Feature	Description
Add Transaction	Add income or expense with title, amount, category, date, and note
Edit Transaction	Update any existing transaction inline using the edit form
Delete Transaction	Remove transactions with a confirmation prompt to prevent accidents
Search	Search transactions by title in real time as the user types
Filter	Filter by type (income or expense) and by category simultaneously
Dashboard Cards	Cards showing total income, total expense, balance, and count
Bar Chart	Visual bar chart comparing total income vs total expense (Recharts)

All features work together in a single-page application without any page reloads. Changes made through add, edit, or delete operations immediately refresh the dashboard cards and chart so the user always sees up-to-date financial information.

Technology Stack

The project uses the MERN stack, which stands for MongoDB, Express.js, React.js, and Node.js. Each technology was chosen for a specific reason and plays a distinct role in the application.

Layer	Technology
Frontend	React.js (Vite), Recharts, Axios
Backend	Node.js, Express.js
Database	MongoDB (Mongoose ODM)
Dev Tools	Nodemon, dotenv

Why These Technologies?

MongoDB was chosen because it stores data in a JSON-like format that integrates naturally with JavaScript. Mongoose adds schema validation on top of MongoDB, preventing bad data from being saved. Express.js provides a minimal and flexible framework for building the REST API with clean routing and middleware support. React.js handles the frontend with component-based architecture and efficient re-rendering through hooks. Vite provides an extremely fast development server with hot module replacement. Axios simplifies HTTP requests from the frontend to the backend. Recharts renders the income versus expense bar chart with smooth animations and tooltips. Nodemon automatically restarts the server whenever a file changes during development.

Folder Structure

The project is divided into two top-level directories. The backend directory contains all server-side code organized using the MVC pattern. The frontend directory contains the React application built with Vite.

```
mern-expense-tracker/
■■■ backend/
■   ■■■ config/
■   ■   ■■■ db.js                  # MongoDB connection
■   ■■■ controllers/
■   ■   ■■■ transaction.controller.js  # Route handlers
■   ■■■ models/
■   ■   ■■■ transaction.model.js   # Mongoose schema
■   ■■■ routes/
■   ■   ■■■ transaction.route.js   # API routes
■   ■■■ .env.example               # Environment variable template
■   ■■■ index.js                   # Express app entry point
■   ■■■ package.json
```

```
■■■ frontend/
    ■■■ src/
    ■   ■■■ services/
    ■   ■   ■■■ api.js            # Axios API calls
    ■   ■■■ App.jsx               # Main React component
    ■   ■■■ main.jsx              # React DOM entry point
    ■   ■■■ style.css             # All styles
    ■■■ index.html
    ■■■ vite.config.js
    ■■■ package.json
```

Inside the backend, the config folder handles the database connection, the models folder defines the Mongoose schema, the controllers folder contains all the business logic for each API operation, and the routes folder maps HTTP methods and URL paths to the correct controller functions. This separation makes the code easier to read, test, and maintain.

Getting Started

Prerequisites

Before running the application, ensure the following are installed:

- Node.js v18 or later — download from nodejs.org
- MongoDB running locally, or a free MongoDB Atlas cluster at mongodb.com/atlas

Step 1 — Clone the Repository

```
git clone https://github.com/Hema-gariki/95.git
cd mern-expense-tracker
```

Step 2 — Set Up the Backend

Open a terminal and run the following commands:

```
cd backend
npm install
copy .env.example .env
```

Edit the .env file and set the following variables:

```
PORT=3000
MONGODB_URI=mongodb://127.0.0.1:27017/expense_tracker
```

For MongoDB Atlas, replace the MONGODB_URI value with your Atlas SRV connection string. Then start the backend server:

```
npm run dev
```

The backend runs at: http://localhost:3000

Step 3 — Set Up the Frontend

Open a new terminal window (keep the backend terminal running) and run:

```
cd frontend
npm install
npm run dev
```

The frontend runs at: http://localhost:5173

Both terminals must remain open while using the application. The backend handles data storage and the frontend handles the user interface. Closing either window will stop that part of the system.

API Endpoints

All endpoints are prefixed with /api/transactions. The server must be running on port 3000 for these endpoints to be accessible.

Method	Endpoint	Description
POST	/api/transactions	Create a new transaction
GET	/api/transactions	Get all transactions (supports filters)
GET	/api/transactions/summary	Get income/expense totals and balance
GET	/api/transactions/:id	Get a single transaction by ID
PUT	/api/transactions/:id	Update a transaction by ID
DELETE	/api/transactions/:id	Delete a transaction by ID

Query Parameters for GET /api/transactions

The GET endpoint that returns all transactions supports optional query parameters for filtering:

Parameter	Values	Description
type	income, expense	Filter by transaction type
category	food, salary, etc.	Filter by category
search	any string	Search by title (case-insensitive)

Example Transaction JSON

When creating or updating a transaction via the POST or PUT endpoints, send a JSON body with the following structure. All fields except note are required.

```
{
  "title": "Grocery",
  "amount": 45.00,
  "type": "expense",
  "category": "food",
  "date": "2026-04-28",
  "note": "Weekly groceries"
}
```

Supported Categories

The category field must be one of the values listed below. The allowed categories are split by transaction type to make classification meaningful:

Expense categories: food, transport, housing, entertainment, healthcare, education, shopping, other

Income categories: salary, freelance, investment, other

Testing the API

The API can be tested using Postman or curl. Below are curl examples for each operation. Replace the ID in the URL with an actual MongoDB ObjectId from a previous response.

Add a transaction (POST)

```
curl -X POST http://localhost:3000/api/transactions \
  -H "Content-Type: application/json" \
  -d '{"title":"Salary","amount":3000,"type":"income","category":"salary","date":"2026-04-01"}'
```

Get all transactions (GET)

```
curl http://localhost:3000/api/transactions
```

Get summary totals (GET)

```
curl http://localhost:3000/api/transactions/summary
```

Filter by type (GET)

```
curl "http://localhost:3000/api/transactions?type=expense"
```

Search by title (GET)

```
curl "http://localhost:3000/api/transactions?search=grocery"
```

How It Works

The application follows a straightforward request-response cycle connecting the browser, the Express server, and the MongoDB database.

1. MongoDB stores all transactions

Every transaction the user adds is saved as a document in a MongoDB collection called transactions. Mongoose enforces the schema, validating that required fields are present and that enum values such as type and category contain only allowed values before the document is written to the database.

2. Express.js exposes the REST API

The Express backend provides six RESTful endpoints covering all CRUD operations. It also includes a special /summary endpoint that uses MongoDB's aggregation pipeline to compute total income, total expense, and the resulting balance in a single database query rather than loading all records to the server and summing them in JavaScript.

3. React fetches data using Axios

The React frontend uses a centralized Axios instance in api.js to send HTTP requests to the backend. When the app loads or when filters change, React calls the backend and updates its state with the returned data. The dashboard cards and chart re-render automatically whenever the state changes.

4. Vite proxy removes CORS issues

During development, Vite's built-in proxy forwards all requests starting with /api from the frontend server at port 5173 to the backend server at port 3000. This means the browser sees all requests

going to the same origin, completely avoiding cross-origin resource sharing restrictions without any extra configuration.

Conclusion

The MERN Expense Tracker is a fully functional full-stack web application that demonstrates the core concepts of the MERN stack. It covers backend API development with Express.js, database design and validation with MongoDB and Mongoose, frontend state management with React hooks, data visualization with Recharts, and HTTP communication with Axios.

The project is version-controlled with Git and hosted on GitHub. The source code is organized cleanly following the MVC pattern on the backend and a component-service pattern on the frontend. The README provides complete setup instructions so anyone can clone the repository and run the application locally within minutes.

This project was completed as a MERN stack web development assignment, submitted on April 30, 2026. The GitHub repository can be found at https://github.com/Hema-gariki/95.

End of Document